

Project Work

Partial Differential Equations and the
Finite Element Method

Agenorwoth Samuel



January 12, 2024

Motivation

The initial presentation of my Practical Placement results on November 15, 2023, led to the realization that I had incorporated an artificial boundary on the right-hand side of my model. In response to this, the examiner for Practical Placement and as well as course instructor for PDEs Professor Eddie Wadbro proposed a project related to this topic, focusing on managing such absorbing boundary conditions, which brings us to the topic of our current discussion.

Introduction

Introducing artificial boundaries is crucial for restricting the scope of computational areas. This approach also aids in reducing the unnatural reflections that happen at these boundaries. In particular, one would like to minimize the amplitudes of waves reflected from these artificial boundaries.

In this work, we introduce the theoretical highly absorbing boundary conditions for first order symmetric systems, followed by highly absorbing boundary conditions for the linearized shallow water equation and then some numerical simulations of the scalar wave equation.

Highly Absorbing Boundary Conditions for First Order Symmetric systems

We consider a general variable coefficient first order strictly hyperbolic systems of the form

$$\frac{\partial u}{\partial t} = \left(A_1(x, y) \frac{\partial}{\partial x} + A_2(x, y) \frac{\partial}{\partial y} + B(x, y) \right) u \quad \text{for } x \geq 0 \quad (1)$$

where A_1, A_2 are symmetric $m \times m$ matrices

To design highly absorbing local boundary conditions for the system in (1), we assume that A_1 has the form

$$A_1 = \begin{pmatrix} A_1^1 & 0 \\ 0 & A_1^2 \end{pmatrix} \text{ where } A_1^1 = \begin{pmatrix} \lambda_1 & & 0 \\ & \ddots & \\ 0 & & \lambda_k \end{pmatrix} \text{ with } \lambda_i < 0 \quad (2)$$

and

$$A_1^2 = \begin{pmatrix} \lambda_{k+1} & & 0 \\ & \ddots & \\ 0 & & \lambda_n \end{pmatrix} \text{ with } \lambda_{k+j} > 0 \quad (3)$$

As a consequence for strict hyperbolicity $\lambda_i \neq \lambda_j$ for $i \neq j$.

In brief, we are assured of distinct and real eigenvalues.

Theoretical Nonlocal Perfectly Absorbing Boundary Condition

We rewrite (1) as

$$\frac{\partial u}{\partial t} = A_1(x, y) \frac{\partial u}{\partial x} + A_2(x, y) \frac{\partial u}{\partial y} + B(x, y)u \quad \text{for } x \geq 0 \quad (4)$$

We multiply through by (4) by $A_1^{-1}(x, y)$ to yield

$$A_1^{-1}(x, y) \frac{\partial u}{\partial t} = \frac{\partial u}{\partial x} + A_1^{-1}(x, y) A_2(x, y) \frac{\partial u}{\partial y} + A_1^{-1}(x, y) B(x, y)u \quad (5)$$

Let $E = -A_1^{-1}A_2$ and $\tilde{B} = -A_1^{-1}B$ then (5) becomes

$$A_1^{-1}(x, y) \frac{\partial u}{\partial t} = \frac{\partial u}{\partial x} - E \frac{\partial u}{\partial y} - \tilde{B}u \quad \text{for} \quad (6)$$

We rewrite (6) to yield

$$\frac{\partial u}{\partial x} = A_1^{-1}(x, y) \frac{\partial u}{\partial t} + E \frac{\partial u}{\partial y} + \tilde{B}u \quad (7)$$

$$\frac{\partial u}{\partial x} = \left(A \frac{\partial u}{\partial t} + E \frac{\partial u}{\partial y} + \tilde{B}u \right) \quad (8)$$

where $A = A_1^{-1} = \begin{pmatrix} \Lambda_1 & 0 \\ 0 & \Lambda_2 \end{pmatrix}$ with $\Lambda_1 < 0$ and $\Lambda_2 > 0$ and we define $M(\xi, \omega)$ by $M(\xi, \omega) = Ai\xi + Ei\omega$

The theoretical construction proceeds by choosing a smooth matrix $V(\xi, \omega, x, y)$ invertible for all (ξ, ω) satisfying $\frac{|\omega|}{|\xi|} + |\omega| < C_0$ for some choice C_0 and defining the symbol of a pseudo-differential operator so that if

$$w = V \left(\frac{\partial}{\partial t}, \frac{\partial}{\partial y}, x, y \right) u$$

then the equation in (8) assume the form

$$\frac{\partial w}{\partial x} = \begin{pmatrix} \Lambda_{11} \left(\frac{\partial}{\partial t}, \frac{\partial}{\partial y}, x, y \right) & 0 \\ \Lambda_{21} \left(\frac{\partial}{\partial t}, \frac{\partial}{\partial y}, x, y \right) & \Lambda_{22} \left(\frac{\partial}{\partial t}, \frac{\partial}{\partial y}, x, y \right) \end{pmatrix} w \quad (9)$$

where Λ_{11} , Λ_{21} and Λ_{22} are matrix pseudo-differential operators of order 1 with dimensions $k \times k$, $(m - k) \times k$ and $(m - k) \times (m - k)$ respectively.

We can evaluate Λ_{11} at $(0, 1, x, y)$, given by

$$\Lambda_{11}(0, 1, x, y) = \Lambda_1 = \text{diag}(\lambda_1^{-1}, \dots, \lambda_k^{-1})$$

and satisfying $\Lambda_{11} < 0$ for (ξ, ω) with $\frac{|\omega|}{|\xi|} + |\omega| < C_0$. The theoretical absorbing boundary condition annihilates the components of Λ_{11}^1 for $x = 0$.

Let $\pi_k(w_1, \dots, w_k, w_{k+1}, \dots, w) \equiv (w_1, \dots, w_k)$. The perfect absorbing condition is given by

$$\pi_k w|_{x=0} = \pi_k V u|_{x=0} = 0. \quad (10)$$

To gain an insight into the nature of this nonlocal condition, we recall Taylor's construction of

$$\begin{aligned} V(\xi, \omega, x, y) &\cong V_0(\xi, \omega, x, y) + \xi^{-1} V_{-1}(\xi, \omega, x, y) \\ &+ \xi^{-2} V_{-2}(\xi, \omega, x, y) + \dots \end{aligned} \quad (11)$$

each V_{-j} is homogeneous of degree zero and $\cong$ is the asymptotic sense of pseudo-differential operators. We can write $M(\xi, \omega)$ as a lower block triangular

$$\begin{pmatrix} {}_1\Lambda_{11} & 0 \\ {}_1\Lambda_{21} & {}_1\Lambda_{22} \end{pmatrix} \equiv V_0 M V_0^{-1} \quad (12)$$

We set $w_0 = V_0 u$, then w_0 formally satisfies the equation in (9) up to error terms which at the symbol level are bounded in ξ .

We write $w_1 = (1 + K_1)w_0$ with $K_1 V_0 = \xi^{-1} V_{-1}$. Using the composition formulas for pseudo-differential operators, we obtain that

$$\begin{aligned} &(1 + K_1) V_0 (M(\xi, \omega) + \tilde{B}) V_0^{-1} (1 + K_1)^{-1} \\ &= K_1 V_0 M(\xi, \omega) V_0^{-1} V_0 M(\xi, \omega) V_0^{-1} K_1 + V_0 (\tilde{B} - \partial_x V_0) V_0^{-1} + \mathcal{O}(\xi^{-1}) \end{aligned} \quad (13)$$

We define $\tilde{C} \cong \tilde{B} - \partial_x V_0$. We choose K_1 to have the form

$$K_1 = \begin{pmatrix} 0 & \tilde{K} \\ 0 & 0 \end{pmatrix} \quad (14)$$

such that the terms in the upper right-hand corner of (9) which are bounded in ξ will vanish and note that $\tilde{K}$ is an $k \times m - k$ matrix satisfying

$$\tilde{K} {}_1\Lambda_{22} - {}_1\Lambda_{11} K_1 + \tilde{C}_{12} = 0 \quad (15)$$

A unique choice of $\tilde{K}$ satisfying the equations in (15) exists provided that ${}_1\Lambda_{22} \equiv \Lambda_2$ and ${}_1\Lambda_{11} = \Lambda_1$ have disjoint spectra.

Highly Absorbing Boundary Conditions for Linearized Shallow water Equations

We consider a system in the half-space $x, t \geq 0$ of the form

$$\frac{\partial u}{\partial t} = \begin{pmatrix} a & 0 & c \\ 0 & a & 0 \\ c & 0 & a \end{pmatrix} \frac{\partial u}{\partial x} + \begin{pmatrix} b & 0 & 0 \\ 0 & b & c \\ 0 & c & b \end{pmatrix} \frac{\partial u}{\partial y} + \begin{pmatrix} 0 & f & 0 \\ -f & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} u \quad (16)$$

with physical restrictions, $0 < a^2 + b^2 < c^2, c > 0$. For simplicity, we shall assume all matrices in (16) are constant implying their elements do not change with respect to any variables or over time.

We then aim to diagonalize the normal matrix to $x = 0$ by using the unitary map, $U(UU^T = U^T U = \mathbb{I})$ where U^T represents U transpose. So if A is a normal matrix then $A = UDU^T$ and the diagonal elements of D are the eigenvalues of A to $x = 0$.

Consider a unitary U as

$$U = \begin{pmatrix} \frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \\ 0 & 1 & 0 \\ -\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} \end{pmatrix} \quad (17)$$

we set $w = U^{-1}u$ and substitute it in (16) thus

$$\begin{aligned} U \frac{\partial w}{\partial t} &= \begin{pmatrix} a & 0 & c \\ 0 & a & 0 \\ c & 0 & a \end{pmatrix} U \frac{\partial w}{\partial x} + \begin{pmatrix} b & 0 & 0 \\ 0 & b & c \\ 0 & c & b \end{pmatrix} U \frac{\partial w}{\partial y} + \begin{pmatrix} 0 & f & 0 \\ -f & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} U w \\ \frac{\partial w}{\partial t} &= U^{-1} \begin{pmatrix} a & 0 & c \\ 0 & a & 0 \\ c & 0 & a \end{pmatrix} U \frac{\partial w}{\partial x} + U^{-1} \begin{pmatrix} b & 0 & 0 \\ 0 & b & c \\ 0 & c & b \end{pmatrix} U \frac{\partial w}{\partial y} + U^{-1} \begin{pmatrix} 0 & f & 0 \\ -f & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} U w \end{aligned} \quad (18)$$

Since U is unitary, then $U^T = U^{-1}$. We can use its transpose without computing U^{-1} to simplify (18) as

$$\begin{aligned} \frac{\partial w}{\partial t} &= \begin{pmatrix} a-c & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & a+c \end{pmatrix} \frac{\partial w}{\partial x} + \begin{pmatrix} b & -\frac{c}{\sqrt{2}} & 0 \\ -\frac{c}{\sqrt{2}} & b & \frac{c}{\sqrt{2}} \\ 0 & \frac{c}{\sqrt{2}} & b \end{pmatrix} \frac{\partial w}{\partial y} \\ &+ \begin{pmatrix} 0 & \frac{f}{\sqrt{2}} & 0 \\ -\frac{f}{\sqrt{2}} & 0 & -\frac{f}{\sqrt{2}} \\ 0 & \frac{f}{\sqrt{2}} & 0 \end{pmatrix} w. \end{aligned} \quad (19)$$

We now aim to (19) inform of (8) using the same procedure with $A = A_1^{-1}, E = -A_1^{-1}A_2$ and $B = -A_1^{-1}B$. The results yields

$$\begin{aligned} \frac{\partial w}{\partial x} &= \begin{pmatrix} \frac{1}{a-c} & 0 & 0 \\ 0 & \frac{1}{a} & 0 \\ 0 & 0 & \frac{1}{a+c} \end{pmatrix} \frac{\partial w}{\partial t} + \begin{pmatrix} \frac{-b}{a-c} & \frac{c}{(a-c)\sqrt{2}} & 0 \\ \frac{c}{a\sqrt{2}} & \frac{-b}{a} & -\frac{c}{a\sqrt{2}} \\ 0 & -\frac{c}{(a+c)\sqrt{2}} & b \end{pmatrix} \frac{\partial w}{\partial y} \\ &+ \begin{pmatrix} 0 & -\frac{f}{(a-c)\sqrt{2}} & 0 \\ \frac{f}{a\sqrt{2}} & 0 & \frac{f}{a\sqrt{2}} \\ 0 & -\frac{f}{(a+c)\sqrt{2}} & 0 \end{pmatrix} w \end{aligned} \quad (20)$$

From (20), the boundary conditions we choose depend upon whether we have linearized about an in flowing state with $a < 0$ or about an out flowing state $a > 0$.

An inflow and outflow case can be studied exclusively with the knowledge of perturbation theory.

Difference Approximation for Absorbing Boundary Conditions - Numerical Experiment for the Scalar Wave Equation

The scalar wave equation is given by

$$w = \frac{\partial^2 w}{\partial t^2} - \frac{\partial^2 w}{\partial x^2} - \frac{\partial^2 w}{\partial y^2} = 0 \quad (21)$$

We start by writing (21) in an approximated form. To do this, we approximate the second derivatives in time t , space x and y using central differences as

$$\frac{\partial^2 w}{\partial t^2} \approx \frac{w(t + \Delta t) - 2w(t) + w(t - \Delta t)}{\Delta t^2} \quad (22)$$

$$\frac{\partial^2 w}{\partial x^2} \approx \frac{w(x + \Delta x) - 2w(x) + w(x - \Delta x)}{\Delta x^2}$$

$$\frac{\partial^2 w}{\partial y^2} \approx \frac{w(y + \Delta y) - 2w(y) + w(y - \Delta y)}{\Delta y^2} \quad (23)$$

But since we aim to approximate the solution $w(t, x, y)$ on the grid $\{(t^n, x_j, y_k)\}$ by the mesh function

$$w_{j,k}^n(t^n = n\Delta t, \quad x_j = j\Delta x - \frac{\Delta x}{2}, \quad y_k = k\Delta y; \\ n = 0, 1, \dots, \quad j = 0, 1, \dots \text{ and } k = \dots, -1, 0, 1, \dots)$$

where $w_{j,k}^n$ represents the value of the function w at the n -th time step and spatial grid points j and k .

We then write (22)–(23) in notation of finite difference formula as

$$\frac{\partial^2 w}{\partial t^2} \approx D_+^t D_-^t w_{j,k}^n = \frac{w_{j,k}^{n+1} - 2w_{j,k}^n + w_{j,k}^{n-1}}{\Delta t^2} \quad (24)$$

$$\frac{\partial^2 w}{\partial x^2} \approx D_+^x D_-^x w_{j,k}^n = \frac{w_{j+1,k}^n - 2w_{j,k}^n + w_{j-1,k}^n}{\Delta x^2} \\ \frac{\partial^2 w}{\partial y^2} \approx D_+^y D_-^y w_{j,k}^n = \frac{w_{j,k+1}^n - 2w_{j,k}^n + w_{j,k-1}^n}{\Delta y^2} \quad (25)$$

where D_+ , D_- and $D_0 = \frac{1}{2}(D_+ + D_-)$ denotes forward, backward and centered divided differences respectively. We finally combine (24)–(25) and write (21) in an approximate form as

$$(D_+^t D_-^t - D_+^x D_-^x - D_+^y D_-^y) w_{j,k}^n = 0 \quad (26)$$

We now discretize the first approximation $(\frac{\partial}{\partial x} - \frac{\partial}{\partial t}) w \Big|_{x=0} = 0$ of the boundary condition to obtain

$$D_+^x (w_{0,k}^n + w_{0,k}^{n+1}) - D_+^t (w_{0,k}^n + w_{1,k}^n) = 0$$

In our case we shall fix values for $w_{j,k}^0$ and $w_{j,k}^1$ as a Gaussian function centered around the point $(5, 5)$ in the spatial domain such that $w_{j,k}^0 = w_{j,k}^1$.

Numerical Simulations for first approximation

Using Dirichlet at the boundary $x = 0$, we use the fashion $w_{0,k}^{n+1} = 0$. The results are obtained are

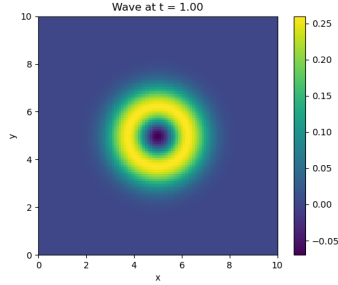


Figure 1: Wave time t=1

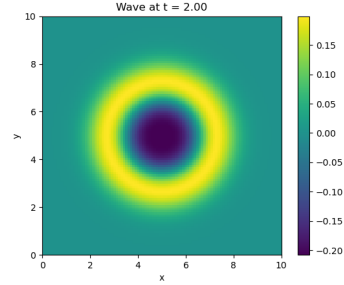


Figure 2: Wave time t=2

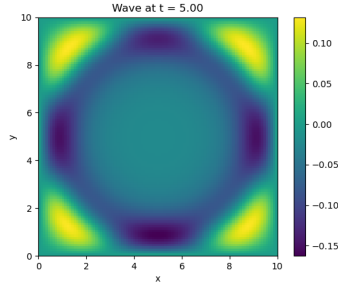


Figure 3: Wave time t=5

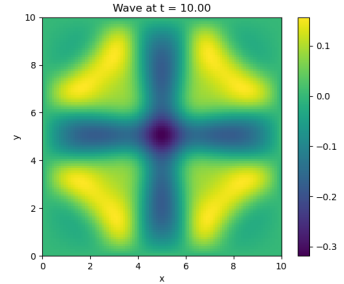


Figure 4: Wave time t=10

For Neumann boundary condition, we use the fashion $D_-^x w_{0,k}^{n+1} = 0$ which implies

$$\frac{w_{0,k}^{n+1} - w_{-1,k}^{n+1}}{\Delta x} = 0$$

We can set $w_{0,k}^{n+1} = w_{-1,k}^{n+1}$ but since our grid starts from $j = 1$, then we can approximate the condition as $w_{0,k}^{n+1} = w_{1,k}^{n+1}$

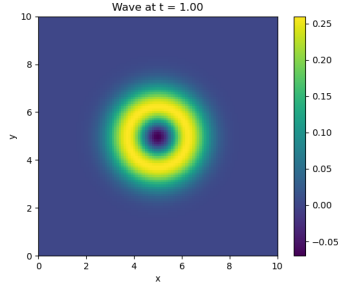


Figure 5: Wave time t=1

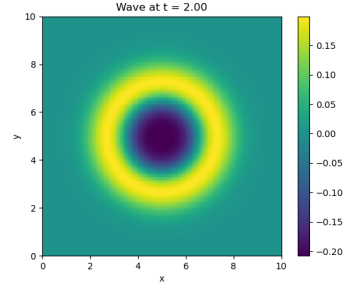


Figure 6: Wave time t=2

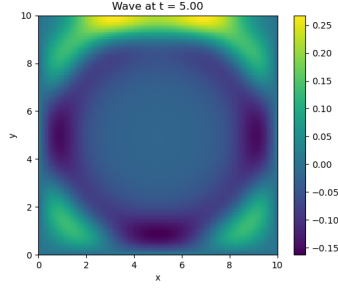


Figure 7: Wave time t=5

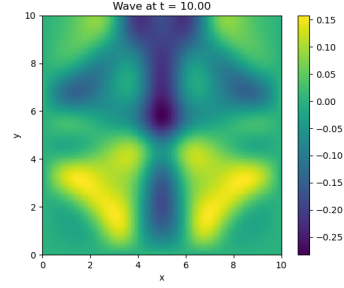


Figure 8: Wave time t=10

For the second highly absorbing boundary $\left(\frac{\partial^2}{\partial x \partial t} - \frac{\partial^2}{\partial t^2} + \frac{1}{2} \frac{\partial^2}{\partial y^2} \right) w \Big|_{x=0} = 0$. we discretize to obtain

$$D_0^t D_t^x w_{0,k}^n - \frac{1}{2} D_+^t D_-^t (w_{0,k}^n + w_{1,k}^n) + \frac{1}{4} D_+^y D_-^y (w_{0,k}^{n-1} + w_{1,k}^{n+1}) = 0$$

It is also important to note that second approximation is a highly absorbing boundary condition compared to the first approximation.

References

- [1] Björn Engquist and Andrew Majda. “Absorbing boundary conditions for numerical simulation of waves”. In: *Proceedings of the National Academy of Sciences* 74.5 (1977), pp. 1765–1766.

Appendix 1

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Defining Parameters
6 max_t = 10 # maximum time
7 max_x = max_y = 10.0 # spatial domain
8 dt = 0.01 # time step
9 dx = dy = 0.1 # spatial step
10 num_t = int(max_t / dt) + 1 # number of time steps
11 num_x = int(max_x / dx) + 1 # number of x steps
12 num_y = int(max_y / dy) + 1 # number of y steps
13
14 # Initialize the grid
15 w = np.zeros((num_t, num_x, num_y))
16
17 # Defining a fixed Initial condition with Gaussian
    pulse
18 for j in range(num_x):
19     for k in range(num_y):
20         x = j * dx - dx / 2
21         y = k * dy
22         w[0, j, k] = np.exp(-((x-5)**2 + (y-5)**2))
23
24 # Copying the initial condition to the first time step
25 w[1, :, :] = w[0, :, :]
26
27 # Define Updating Wave Function with Finite difference
    method
28 for n in range(1, num_t-1):
29     for j in range(1, num_x-1):
30         for k in range(1, num_y-1):
31             w[n+1, j, k] = (2 * w[n, j, k] - w[n-1, j,
32                                     k] +
33                             dt**2 * ((w[n, j+1, k] -
34                                         2*w[n, j, k] + w[n, j
35                                         -1, k]) / dx**2 +
36                                         (w[n, j, k+1] -
37                                             2*w[n, j, k] +
38                                             w[n, j, k-1]) /
39                                             dy**2))
34
35 # Apply Dirichlet boundary condition at x = 0
36 w[n+1, 0, :] = 0
```

```

37
38 # Plotting the result at the final time step
39 plt.imshow(w[-1, :, :], extent=[0, max_x, 0, max_y])
40 plt.colorbar()
41 plt.title("Wave at t = {:.2f}".format(max_t))
42 plt.xlabel("x")
43 plt.ylabel("y")
44
45 # Save the figure
46 plt.savefig("Wave_simulation t=10.png")
47
48 plt.show()

```

Listing 1: Python code for simulation of wave with Dirichelet

Appendix 2

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Defining Parameters
6 max_t = 10 # maximum time
7 max_x = max_y = 10.0 # spatial domain
8 dt = 0.01 # time step
9 dx = dy = 0.1 # spatial step
10 num_t = int(max_t / dt) + 1 # number of time steps
11 num_x = int(max_x / dx) + 1 # number of x steps
12 num_y = int(max_y / dy) + 1 # number of y steps
13
14 # Initialize the grid
15 w = np.zeros((num_t, num_x, num_y))
16
17 # Defining Initial condition with Gaussian pulse
18 for j in range(num_x):
19     for k in range(num_y):
20         x = j * dx - dx / 2
21         y = k * dy
22         w[0, j, k] = np.exp(-((x-5)**2 + (y-5)**2))
23
24 # Copy initial condition to the first time step
25 w[1, :, :] = w[0, :, :]
26
27 # Define Updating Wave Function with Finite difference
    method
28 for n in range(1, num_t-1):
29     for j in range(1, num_x-1):
30         for k in range(1, num_y-1):
31             w[n+1, j, k] = (2 * w[n, j, k] - w[n-1, j,
32                 k] +
33
34                             dt**2 * ((w[n, j+1, k] -
35                                     2*w[n, j, k] + w[n, j
36                                     -1, k]) / dx**2 +
37
38                                     (w[n, j, k+1] -
39                                     2*w[n, j, k] +
40                                     w[n, j, k-1]) /
41                                     dy**2))
42
43 # Apply Neumann boundary condition at x = 0
44 w[n+1, 0, :] = w[n+1, 1, :]
```

```

38 # Plotting the result at the final time step
39 plt.imshow(w[-1, :, :], extent=[0, max_x, 0, max_y])
40 plt.colorbar()
41 plt.title("Wave at t = {:.2f}".format(max_t))
42 plt.xlabel("x")
43 plt.ylabel("y")
44
45 # Save the figure
46 plt.savefig("Wave_simulation tt=10.png")
47
48 plt.show()

```

Listing 2: Python code for simulation of wave with Neumann